

Statistika a pravděpodobnost (MSP 2025/26Z) – 2. projekt

- **Autor:** David Kvaček (xkvace00@stud.fit.vutbr.cz)
- **Datum:** Brno, 14. prosince 2025

Věrohodnost

Analýza se zabývá dobou, po kterou absolventi VUT pracují ve svém oboru po ukončení studia. Data obsahují zprava cenzorovaná pozorování. Pro dobu zaměstnání předpokládáme Weibullovo rozdělení pravděpodobnosti začínajícím od nuly (parametr prahu, tzv. threshold, je roven nule).

1) Parametrizace Weibullova rozdělení, logaritmická–věrohodnostní funkce a její parciální derivace

Předpokládáme Weibullovo rozdělení s parametry tvaru $\alpha > 0$ a měřítka $\beta > 0$. Hustota pravděpodobnosti (PDF) je dána vztahem:

$$f(x; \alpha, \beta) = \frac{\alpha}{\beta} \left(\frac{x}{\beta} \right)^{\alpha-1} \exp \left[- \left(\frac{x}{\beta} \right)^{\alpha} \right], \quad x \geq 0$$

Věrohodnostní funkce pro n nezávislých pozorování $x_1, x_2, \dots, x_n$ a funkci přežití $S(x) = \exp \left[- \left(\frac{x}{\beta} \right)^{\alpha} \right]$, je dána vztahem:

$$L(\alpha, \beta) = \prod_{i=1}^n [f(x_i; \alpha, \beta)]^{\delta_i} \cdot [S(x_i; \alpha, \beta)]^{1-\delta_i} = \prod_{i=1}^n \left\{ \frac{\alpha}{\beta} \left(\frac{x_i}{\beta} \right)^{\alpha-1} \exp \left[- \left(\frac{x_i}{\beta} \right)^{\alpha} \right] \right\}^{\delta_i} \cdot \left\{ \exp \left[- \left(\frac{x_i}{\beta} \right)^{\alpha} \right] \right\}^{1-\delta_i}$$

Logaritmická–věrohodnostní funkce, kde $\delta_i = 0$ značí cenzorované a $\delta_i = 1$ necenzorované pozorování (tj. pro výpočty $\delta_i = 1 - censored_i$), je tedy definována jako:

$$\ell(\alpha, \beta) = \ln L(\alpha, \beta) = \sum_{i=1}^n \delta_i \left[\ln\left(\frac{\alpha}{\beta}\right) + (\alpha - 1) \ln\left(\frac{x_i}{\beta}\right) \right] - \left(\frac{x_i}{\beta}\right)^\alpha$$

Parciální derivace logaritmicke-věrohodnostní funkce podle parametrů α a β jsou následující:

$$\frac{\partial \ell(\alpha, \beta)}{\partial \alpha} = \sum_{i=1}^n \left[\frac{\delta_i}{\alpha} + \delta_i \ln\left(\frac{x_i}{\beta}\right) - \left(\frac{x_i}{\beta}\right)^\alpha \ln\left(\frac{x_i}{\beta}\right) \right]$$

$$\frac{\partial \ell(\alpha, \beta)}{\partial \beta} = \sum_{i=1}^n \left[-\frac{\delta_i \alpha}{\beta} + \frac{\alpha}{\beta} \left(\frac{x_i}{\beta}\right)^\alpha \right] = \frac{\alpha}{\beta} \sum_{i=1}^n \left[\left(\frac{x_i}{\beta}\right)^\alpha - \delta_i \right]$$

2) Maximálně věrohodný odhad (MLE) parametrů Weibullova rozdělení

```
In [1]: import numpy as np
import pandas as pd
import scipy.stats as stats
import statsmodels.api as sm
from scipy.optimize import minimize

# load likelihood data
df_likelihoood = pd.read_excel('data.xlsx', sheet_name=0)

# extract x and delta columns
x = df_likelihoood.iloc[:, 1].values
delta = df_likelihoood.iloc[:, 0].values

# negative logarithmic likelihood estimation
def weibull_negative_logarithmic_likelihood(parameters: list, x: np.array, delta: np.array) -> float:
    """Estimate the negative logarithmic likelihood for Weibull distribution.

    Args:
        parameters (float[]): shape (alpha) and scale (beta) parameters of Weibull distribution
        x (np.array): observed data
        delta (np.array): censoring indicators
```

```

Returns:
    -logarithmic_likelihood (float): negative logarithmic likelihood value
    ....

alpha, beta = parameters
if alpha <= 0 or beta <= 0:
    return np.inf

logarithmic_likelihood = np.sum((1 - delta) * (np.log(alpha / beta) + (alpha - 1) * np.log(x / beta)) -
return -logarithmic_likelihood

def exponential_negative_logarithmic_likelihood(parameters, x, delta):
    return weibull_negative_logarithmic_likelihood([1.0, parameters[0]], x, delta)

# estimate Weibull parameters
initial_parameters = [1.0, 1.0]
weibull = minimize(weibull_negative_logarithmic_likelihood, initial_parameters, args=(x, delta), method='L-
print(f'Estimated Weibull parameters: shape (alpha) = {weibull.x[0]:.4f}, scale (beta) = {weibull.x[1]:.4f}

```

Estimated Weibull parameters: shape (alpha) = 0.9544, scale (beta) = 2675.2131

Pomocí numerické optimalizace jsou nalezeny maximálně věrohodné odhady parametrů Weibullova rozdělení pro daná data. Výsledné odhady jsou následující:

- parametr tvaru (α): **0.9544**,
- parametr měřítka (β): **2675.2131**.

3) Test hypotézy věrohodnostním poměrem

Pro testování hypotézy $H_0 : \alpha = 1$ (exponenciální rozdělení) proti alternativní hypotéze $H_1 : \alpha \neq 1$ (Weibullovo rozdělení), zda je exponenciální rozdělení vhodným modelem pro zadaná data, je použit test věrohodnostním poměrem.

```
In [2]: exponential = minimize(exponential_negative_logarithmic_likelihood, [1.0], args=(x, delta), method='L-BFGS-
```

```
# likelihood ratio test
logarithmic_likelihood_weibull = -weibull.fun
logarithmic_likelihood_exponential = -exponential.fun
test_statistic = -2 * (logarithmic_likelihood_exponential - logarithmic_likelihood_weibull)
p_value = 1 - stats.chi2.cdf(test_statistic, df=1)
print(f'Likelihood Ratio Test Statistic: {test_statistic:.4f}, p-value: {p_value:.4f}')
```

Likelihood Ratio Test Statistic: 0.8575, p-value: 0.3544

Nezamítáme nulovou hypotézu H_0 na hladině významnosti 5 % a můžeme tedy předpokládat, že exponenciální rozdělení je postačujícím modelem pro zadaná data.

4) Bodové odhady střední doby zaměstnání v oboru a 10 % percentil zaměstnání v oboru

```
In [3]: # mean and 10th percentile estimation for exponential distribution
mean_value_exponential = exponential.x[0]
p10_value_exponential = exponential.x[0] * (-np.log(0.9))
print(f'Estimated Mean Employment Duration: {mean_value_exponential:.4f}')
print(f'Estimated 10th Percentile Employment Duration: {p10_value_exponential:.4f}')
```

Estimated Mean Employment Duration: 2690.7987

Estimated 10th Percentile Employment Duration: 283.5039

Bodové odhady střední doby zaměstnání v oboru a 10 % percentil zaměstnání v oboru jsou vypočteny za předpokladu exponenciálního rozdělení (tj. Weibullovo rozdělení s parametrem tvaru $\alpha = 1$).

- střední doba zaměstnání v oboru: **2690.7987 dní**,
- 10 % percentil zaměstnání v oboru: **283.5039 dní**.

5) Slovní charakteristika fungování doby zaměstnání v oboru jako náhodné veličiny

Doba zaměstnání v oboru je modelována exponenciálním rozdělením. Pravděpodobnost, že absolvent VUT zůstane zaměstnán ve svém oboru déle než určitý počet dní, klesá exponenciálně s rostoucím časem. To znamená, že čím déle je absolvent zaměstnán, tím menší je pravděpodobnost, že v tomto zaměstnání zůstane ještě déle.

Regrese

Analýza se zabývá odezvou aplikace (ping) na základě uživatelských dat.

1) Určení vhodného regresního modelu pomocí zpětné eliminace

Za výchozí model předpokládáme plný kvadratický model se všemi interakcemi druhého řádu a všemi druhými mocninami dávající smysl v kontextu dat.

```
In [4]: # load regression data
df_regression = pd.read_excel('data.xlsx', sheet_name=1)

# create squared features
df_regression['ActiveUsers2'] = df_regression['ActiveUsers'] ** 2
df_regression['InteractingPct2'] = df_regression['InteractingPct'] ** 2
df_regression['ScrollingPct2'] = df_regression['ScrollingPct'] ** 2

# create interaction features
df_regression['ActiveUsersInteractingPct'] = df_regression['ActiveUsers'] * df_regression['InteractingPct']
df_regression['ActiveUsersScrollingPct'] = df_regression['ActiveUsers'] * df_regression['ScrollingPct']
df_regression['InteractingPctScrollingPct'] = df_regression['InteractingPct'] * df_regression['ScrollingPct']

# one-hot encode OStype categorical variable
df_regression = pd.get_dummies(df_regression, columns=['OStype'], drop_first=True)
df_regression.columns = [c.replace(' ', '_') for c in df_regression.columns]

def backward_elimination(data: pd.DataFrame, target: str, significance_level: float = 0.05) -> tuple[sm.OLS, list[str]]:
    """Perform backward elimination for feature selection in regression.

    Args:
        data (pd.DataFrame): input data with features and target variable
        target (str): name of the target variable column
        significance_level (float): significance level for feature removal

    Returns:
```

```

        (sm.regression.linear_model.RegressionResultsWrapper, list): final regression model and selected fe
    """

    features = data.columns.tolist()
    features.remove(target)
    while len(features) > 0:
        x = sm.add_constant(data[features])
        y = data[target]
        model = sm.OLS(y, x.astype(float)).fit()
        p_values = model.pvalues.iloc[1:] # exclude intercept
        maximum_p_value = p_values.max()
        if maximum_p_value > significance_level:
            excluded_feature = p_values.idxmax()
            features.remove(excluded_feature)

        else:
            break

    return model, features

# perform backward elimination
final_model, selected_features = backward_elimination(df_regression, target='Ping_[ms]')
print(final_model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Ping_[ms]    R-squared:                0.816
Model:                  OLS          Adj. R-squared:           0.813
Method:                 Least Squares  F-statistic:             273.5
Date:                  Sun, 14 Dec 2025  Prob (F-statistic):       7.18e-176
Time:                  23:55:01      Log-Likelihood:          -1639.4
No. Observations:      502          AIC:                     3297.
Df Residuals:          493          BIC:                     3335.
Df Model:              8
Covariance Type:       nonrobust

=====

```

	coef	std err	t	P> t	[0.025	0.975]
--	------	---------	---	------	--------	--------

const	10.8370	0.946	11.458	0.000	8.979	12.695
ActiveUsers	0.0057	0.000	16.057	0.000	0.005	0.006
InteractingPct	13.3796	0.767	17.450	0.000	11.873	14.886
ScrollingPct	-2.5426	0.715	-3.558	0.000	-3.947	-1.139
ActiveUsers2	-4.262e-07	4.72e-08	-9.039	0.000	-5.19e-07	-3.34e-07
InteractingPct2	8.6508	1.335	6.480	0.000	6.028	11.274
ScrollingPct2	-7.2714	1.311	-5.549	0.000	-9.846	-4.697
ActiveUsersInteractingPct	0.0015	0.000	5.456	0.000	0.001	0.002
ActiveUsersScrollingPct	0.0042	0.000	16.594	0.000	0.004	0.005
InteractingPctScrollingPct	4.7288	1.407	3.360	0.001	1.963	7.494
OSType_MacOS	9.4692	0.818	11.579	0.000	7.862	11.076
OSType_Windows	3.8006	0.821	4.628	0.000	2.187	5.414
OSType_iOS	-5.4757	0.848	-6.458	0.000	-7.142	-3.810
=====						
Omnibus:	97.875	Durbin-Watson:	1.938			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	500.653			
Skew:	0.740	Prob(JB):	1.93e-109			
Kurtosis:	7.663	Cond. No.	1.10e+24			
=====						

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 8.89e-31. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.